Milestone 1 : Object Foundation and System Modeling

Data Cleaning and Transformation Pipeline

1. Introduction

In modern data-driven environments, raw data is often incomplete, inconsistent, or unstructured. Before meaningful analysis or machine learning processes can be applied, the data must first undergo **cleaning and transformation** to improve its quality and usability.

This project focuses on designing a **Data Cleaning and Transformation Pipeline**, which is a structured system that processes raw input data through a sequence of operations and produces clean, transformed, and usable output data.

The main objective of **Milestone 1** is to establish the **foundation of the system** by:

- Identifying system components
- Modeling the system using object-oriented principles
- Implementing a **minimum working prototype**

At this stage, the system is designed as a simple, linear pipeline without advanced logic or dynamic behavior.

System overview

2The Data Cleaning and Transformation Pipeline is a structured system that processes data through four main stages:

1. **Data Input (Source)**
2. **Data Cleaning**
3. **Data Transformation**
4. **Data Output**

The system follows a **sequential processing flow**, where each stage depends on the output of the previous stage.

At this level, the system is:

- Static (no decision-making)
- Linear (fixed execution or

3. Object-Oriented System Modeling

To represent the system properly, it is decomposed into four core objects. Each object has a specific responsibility, following object-oriented design principles.

3.1 DataSource

- Acts as the entry point of the system
- Responsible for providing raw data
- Simulates or loads dataset from external or internal sources

3.2 Cleaner

- Responsible for preprocessing raw data
- Removes invalid, missing, or inconsistent values (e.g., `None`)
- Ensures data quality before transformation

3.3 Transformer

- Applies transformations to cleaned data
- Example operations include:
    - Scaling values
    - Mathematical operations
    - Format conversion

3.4 OutputHandler

- Responsible for presenting or storing the final processed data
- Acts as the exit point of the system
- Displays results in readable format

4. System Flow

The system follows a **linear pipeline structure**:

```
DataSource → Cleaner → Transformer → OutputHandler
```

Flow Description:

- The **DataSource** provides raw input data
- The **Cleaner** removes invalid or missing values
- The **Transformer** processes the cleaned data
- The **OutputHandler** displays the final result

This flow ensures that data moves step-by-step through the system in a controlled manner.

5. Prototype Implementation

A minimum working object-oriented prototype of the system is implemented below:

```
class DataSource:
    def get_data(self):
        return [10, None, 20, 30, None, 40]


class Cleaner:
    def clean(self, data):
        # Remove missing values
        cleaned_data = [x for x in data if x is not None]
        return cleaned_data


class Transformer:
    def transform(self, data):
        # Example transformation: multiply each value by 2
        return [x * 2 for x in data]


class OutputHandler:
    def output(self, data):
        print("Processed Data:", data)


# System Execution
data_source = DataSource()
```

```
cleaner = Cleaner()

transformer = Transformer()

output_handler = OutputHandler()


# Pipeline Flow

data = data_source.get_data()

cleaned_data = cleaner.clean(data)

transformed_data = transformer.transform(cleaned_data)

output_handler.output(transformed_data)
```

6. Explanation of the Prototype

The prototype demonstrates a working object-oriented pipeline system:

- The **DataSource** generates raw data containing missing values (`None`)
- The **Cleaner** removes all missing values to ensure data integrity
- The **Transformer** applies a simple transformation (multiplying values by 2)
- The **OutputHandler** displays the final processed output

Key Observations:

- Each object has a single responsibility
- Data flows sequentially through the system
- The system is functional and executable
- Object-oriented design is clearly implemented

7. System Design Justification

This design follows key software engineering principles:

- **Encapsulation:** Each class handles its own responsibility
- **Modularity:** System is divided into independent components
- **Reusability:** Objects can be reused in other systems
- **Simplicity:** Easy to understand and extend in future milestones

This foundation allows the system to be expanded into more complex versions in later milestones.

8. Limitations of Milestone 1

At this stage, the system has some limitations:

- No interaction between objects beyond simple data passing
- No decision-making or logic control
- No dynamic behavior or adaptability
- Fixed sequential pipeline only

These limitations will be addressed in Milestone 2 and beyond.

9. Conclusion

Milestone 1 successfully establishes the foundation of the Data Cleaning and Transformation Pipeline using object-oriented principles.

The system is:

- Properly structured into objects
- Functionally working as a pipeline
- Designed with clear responsibilities
- Ready for extension in later milestones

This foundation will be expanded in subsequent milestones to include:

- Object interaction
- Control logic
- Multi-object systems
- Dynamic behavior and adaptability